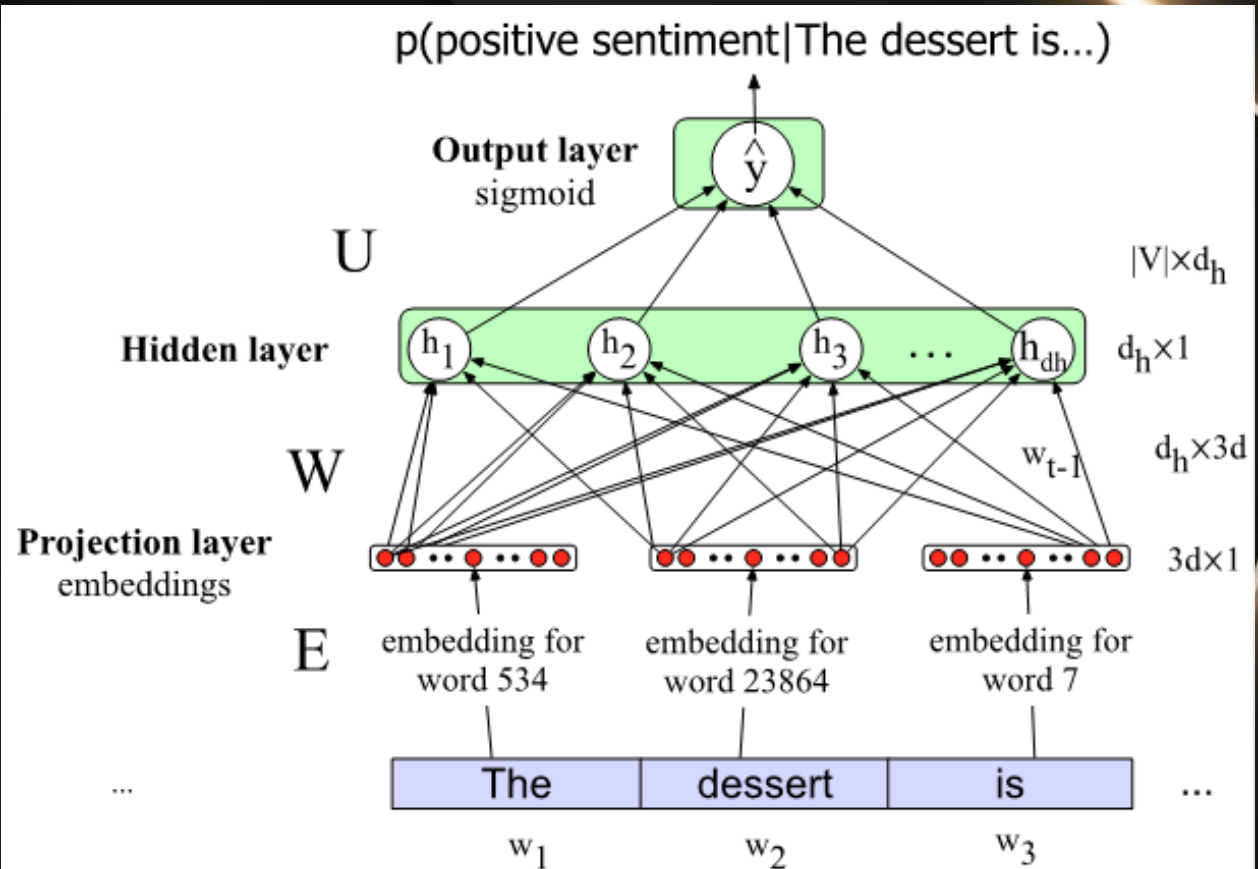




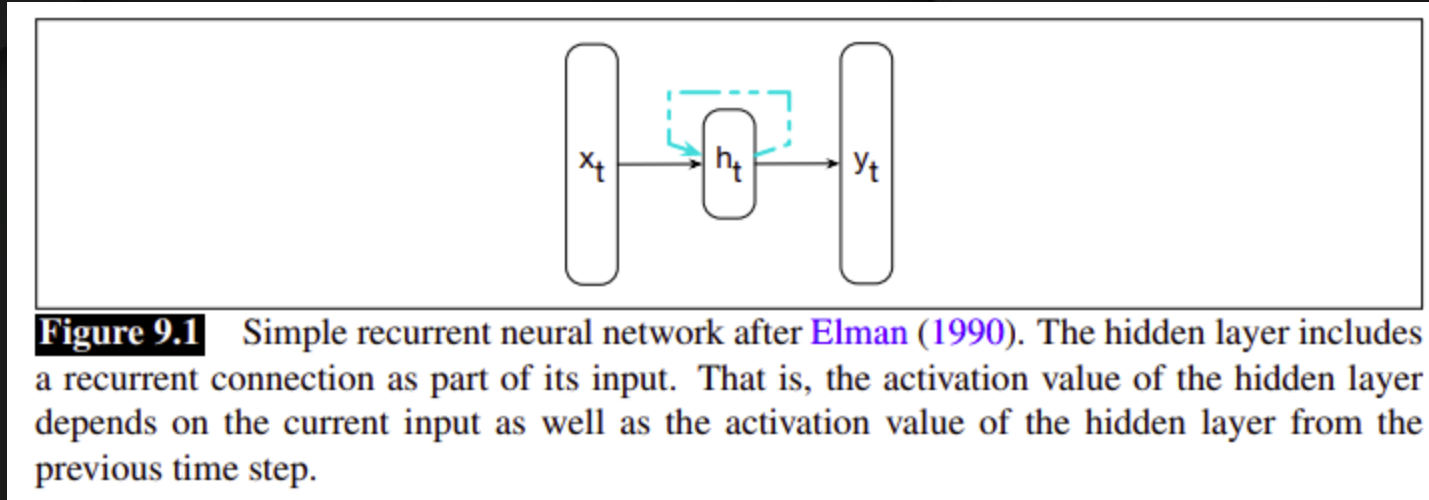
More Deep/Machine Learning NLP Models

Recall Using Feed-Forward NNs for NLP

- How were the input layers set up to incorporate the embeddings?
- What issues could this cause?



Simple Recurrent Neural Network (RNN)



Simple Recurrent Neural Network (RNN)

- Current hidden layer output value depends on the previous hidden layer output value.

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

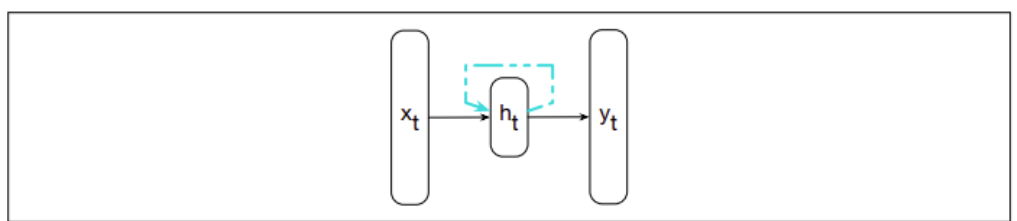


Figure 9.1 Simple recurrent neural network after [Elman \(1990\)](#). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

Simple Recurrent Neural Network (RNN)

- Current hidden layer output value depends on the previous hidden layer output value.

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

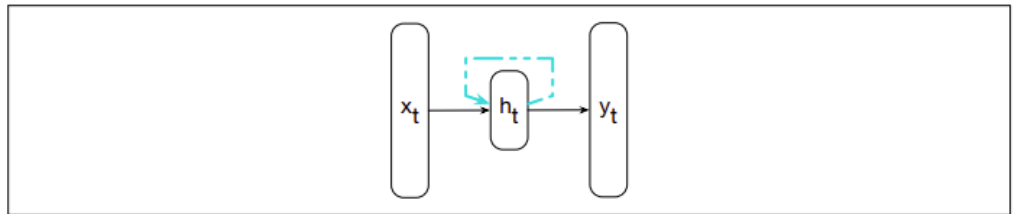


Figure 9.1 Simple recurrent neural network after [Elman \(1990\)](#). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

function FORWARDRNN($\mathbf{x}$, $network$) **returns** output sequence $\mathbf{y}$

$\mathbf{h}_0 \leftarrow 0$

for $i \leftarrow 1$ **to** LENGTH($\mathbf{x}$) **do**

$\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$

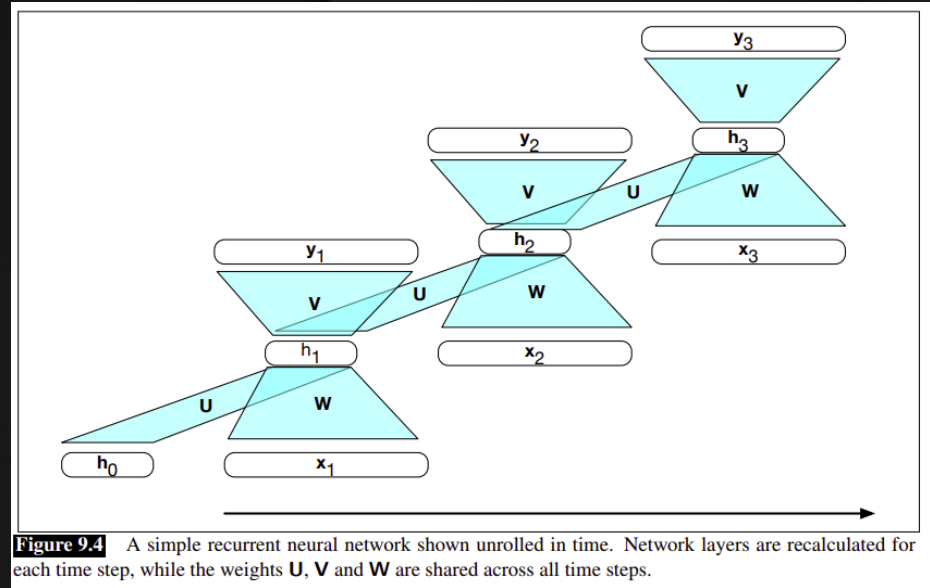
$\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$

return $\mathbf{y}$

Figure 9.3 Forward inference in a simple recurrent network. The matrices $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ are shared across time, while new values for $\mathbf{h}$ and $\mathbf{y}$ are calculated with each time step.

RNNs

- Unrolled representation
- (Encoder-decoder architecture expanded on next week)



RNN for classification

- Entire sequence fed into RNN and final hidden layer used for classification via activation function.

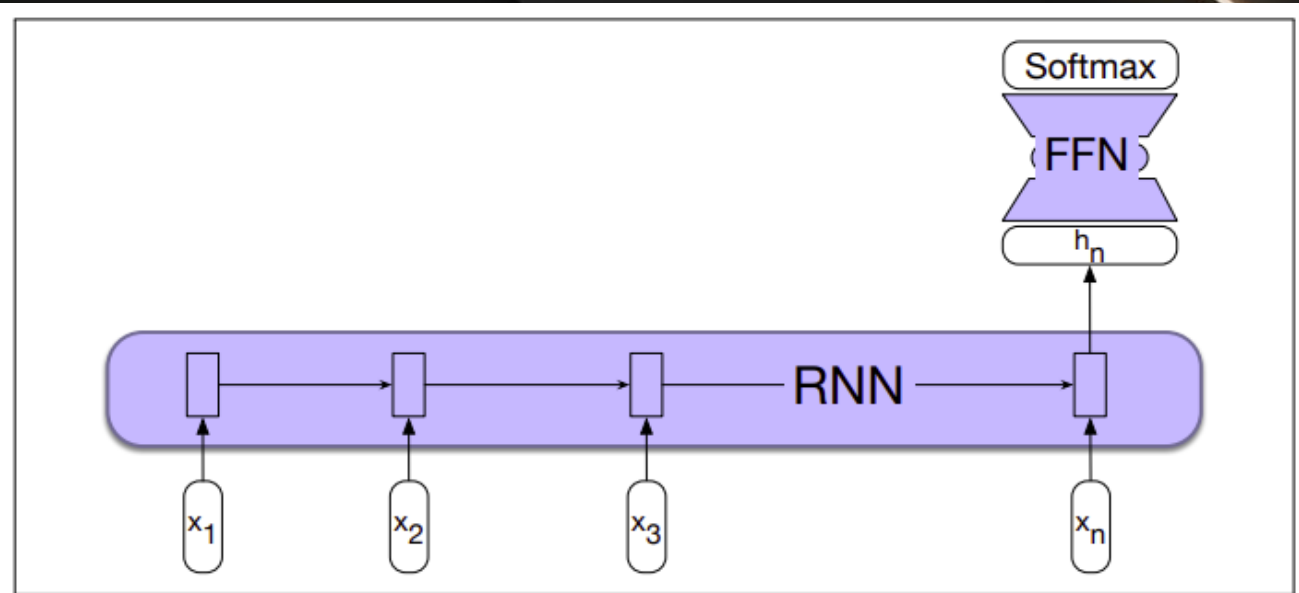


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

RNN for LM/Generation

- Use previous chosen words as input for next word.
- Autoregressive – predict current word from words at previous time steps.

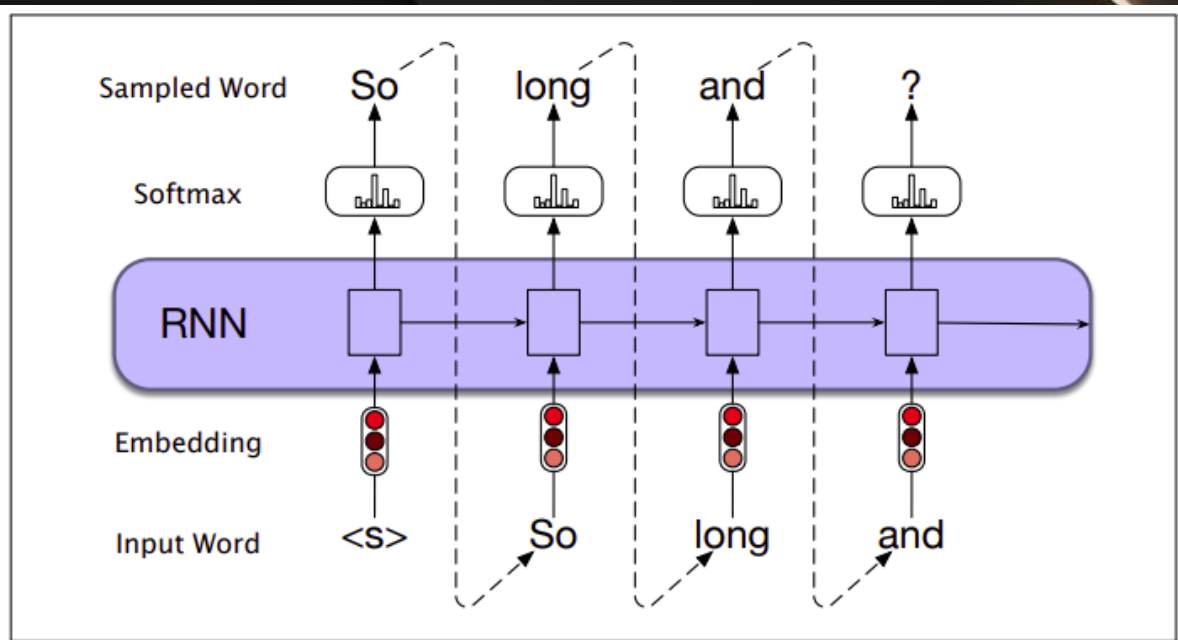


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

Bidirectional RNN

- One layer to read text in left to right and one layer to read text in right to left.
- Why might this be useful?

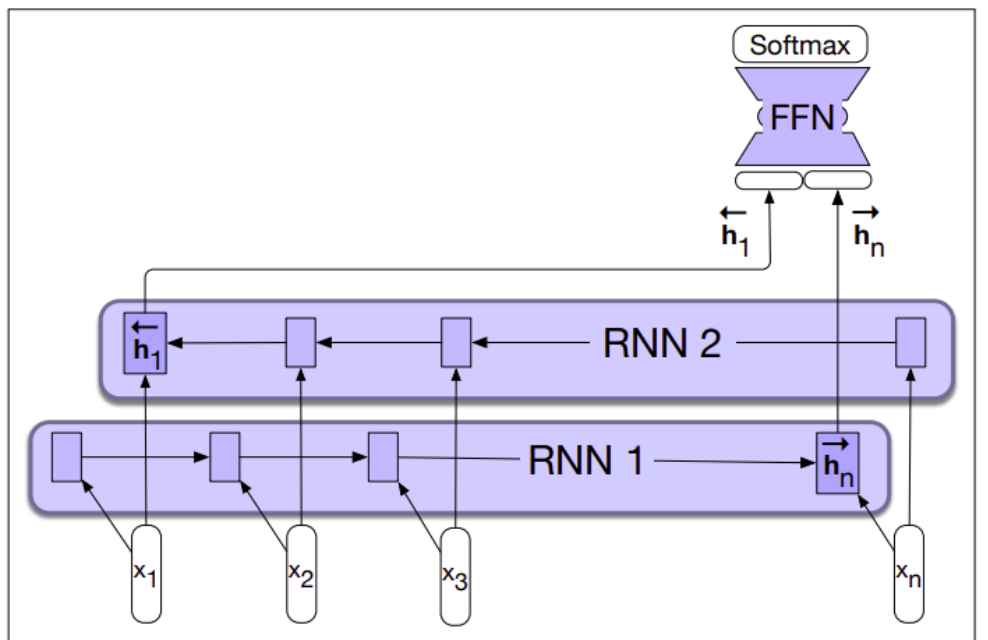
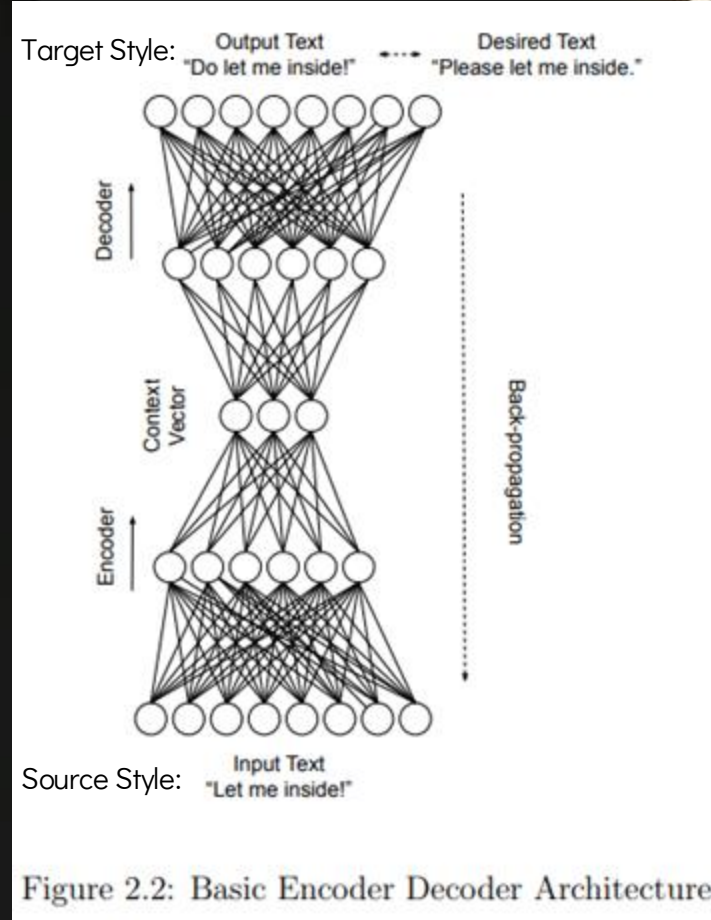


Figure 9.12 A bidirectional RNN for sequence classification. The final hidden units from the forward and backward passes are combined to represent the entire sequence. This combined representation serves as input to the subsequent classifier.

Encoder-Decoder



Training

- Parameters (weights and biases) are randomized when model is first initialized
- Parameters are updated (trained) to produce the desired output
- Training data contains instances of input and desired output
- To update parameters, an error calculated by comparing the desired output to the actual output
- Loss functions used to calculate error

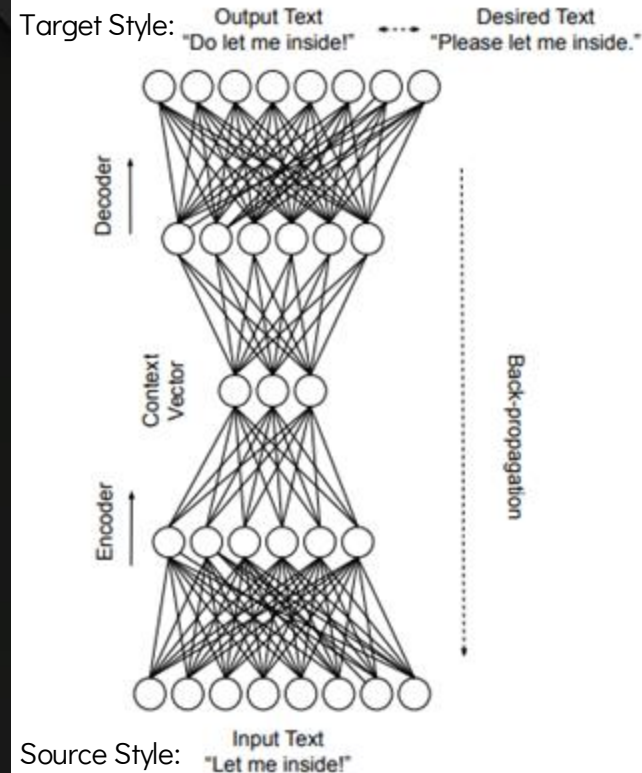
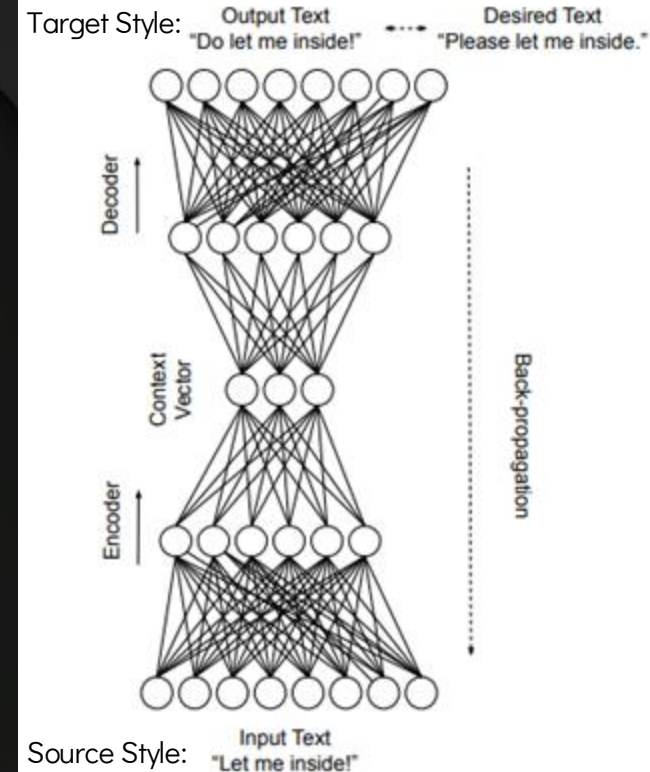


Figure 2.2: Basic Encoder Decoder Architecture

Encoder-Decoder

- Encode input X into a representative state z (the context vector)
- Decode z into output text
- Latent space represents all learned z
- Can have multiple layers
- If parallel data available, original style can be input and target style can be output text
- Otherwise, auto-encoders leveraged:
 - Denoising Auto encoders
 - Variational Auto encoders

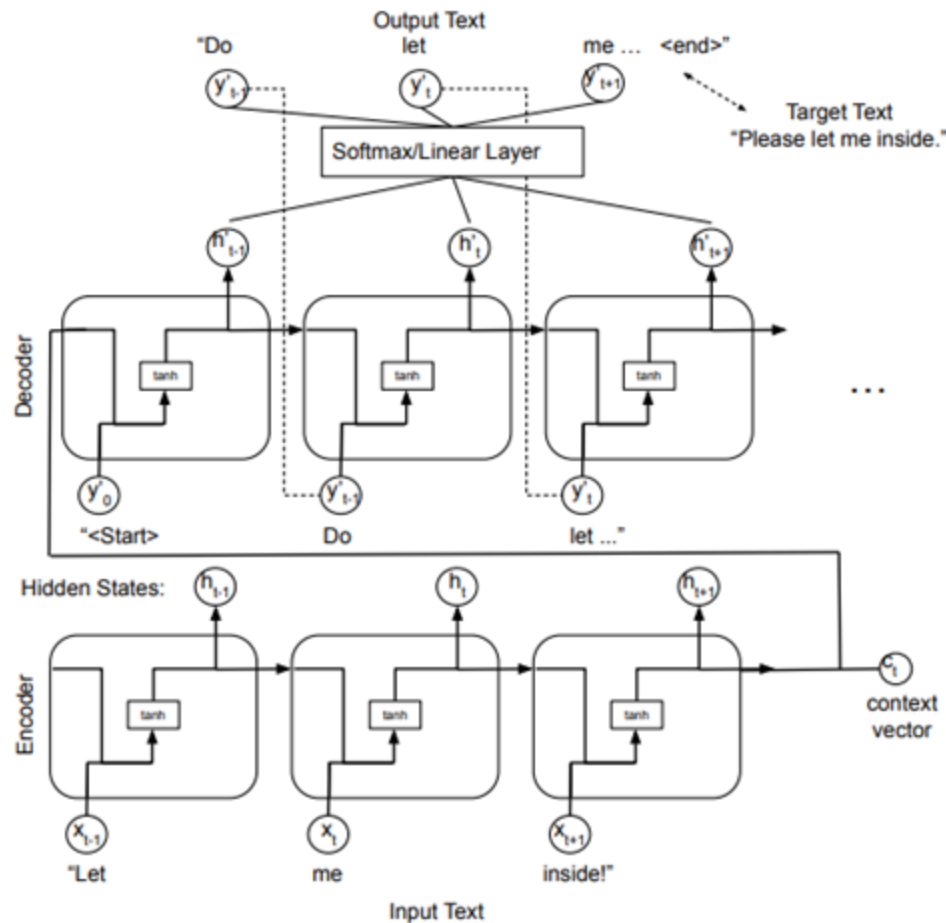


Source Style: "Let me inside!"

Figure 2.2: Basic Encoder Decoder Architecture

RNN Encoder-Decoder

-unrolled representation



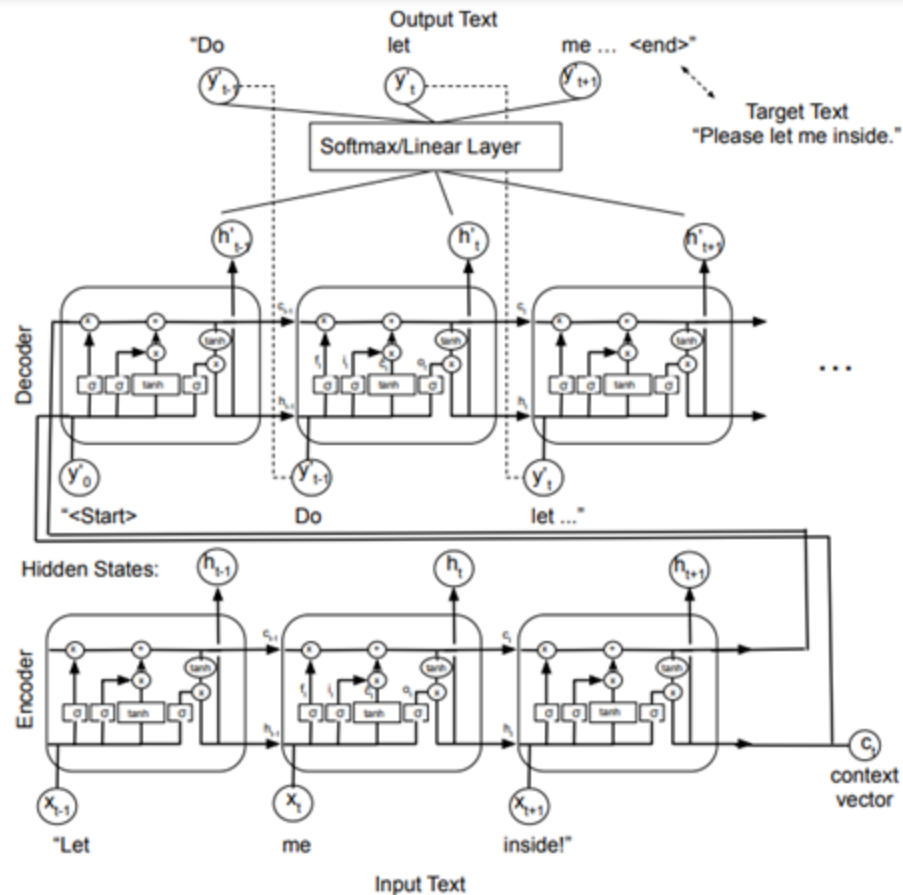
Issues with RNN

- Asked to perform two tasks simultaneously:
 - Provide information useful for current decision
 - Update and carry forward information required for future decisions
- Vanishing Gradient
 - During the backward pass of training, the hidden layers are subject to repeated multiplications, as determined by the length of the sequence. A frequent result of this process is that the gradients are eventually driven to zero.

Long Short- term Memory network (LSTM)

LSTM/GRU

- Adds gates to recurrent unit to extend long term memory



LSTM/GRU

- Forget gate
- Input gate
- Output gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

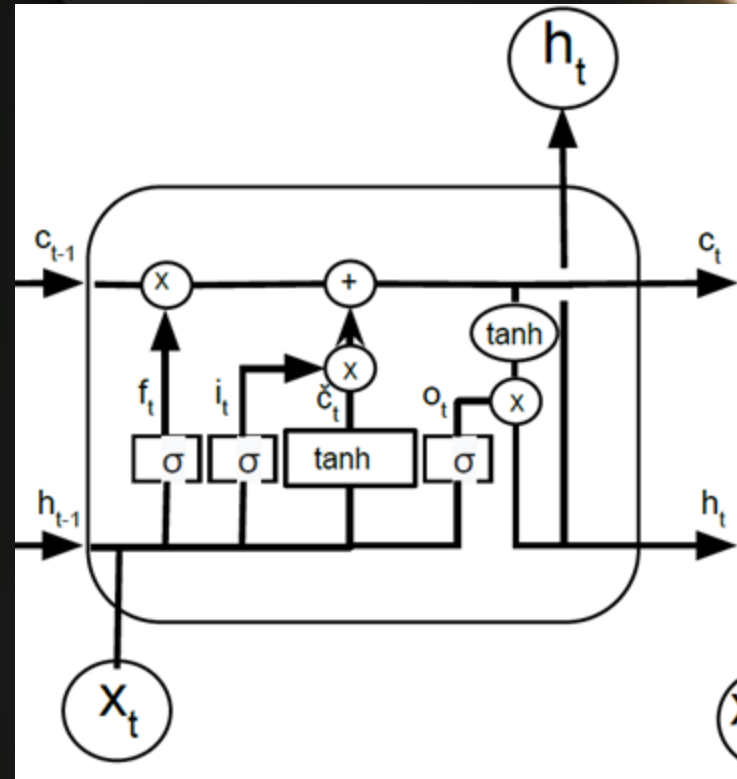
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \hat{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

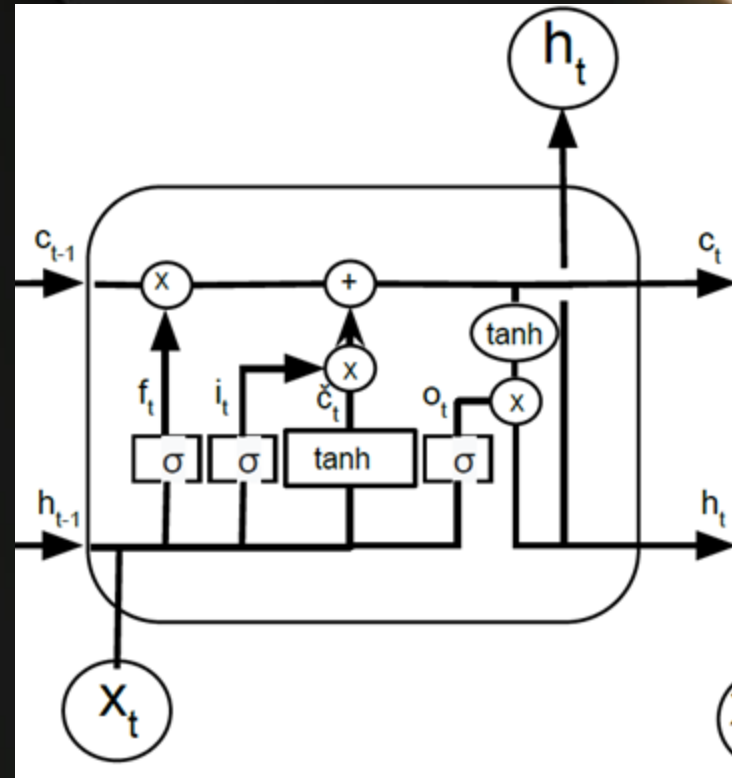


LSTM/GRU

- Forget gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Deletes information that is no longer needed.
- Computes a weighted sum of the previous state's hidden layer and current input and passes through a sigmoid.
- Multiplied by the context vector to remove the irrelevant information.

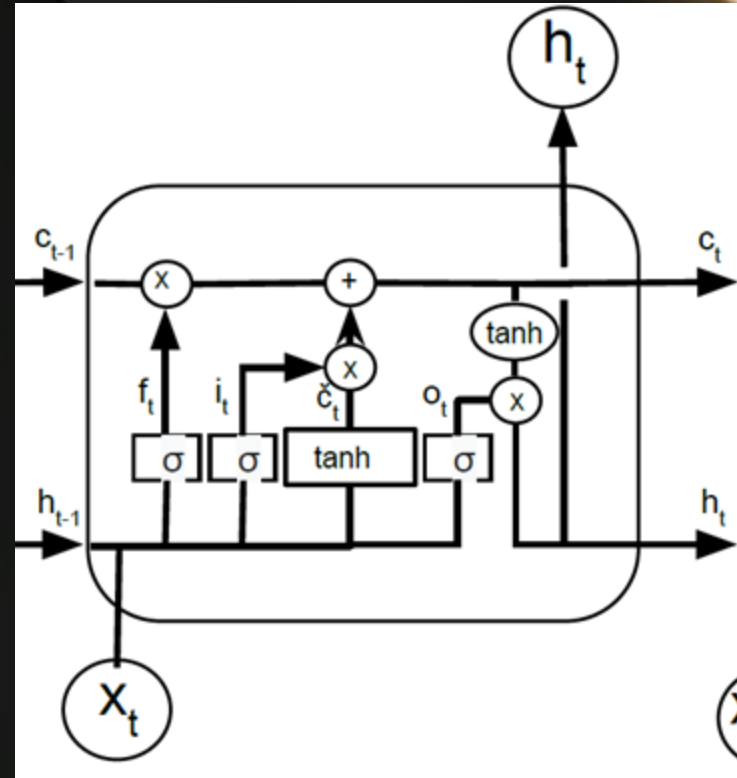


LSTM/GRU

- Input gate/Add gate
 - o Select which information to add to the current context.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{c}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



LSTM/GRU

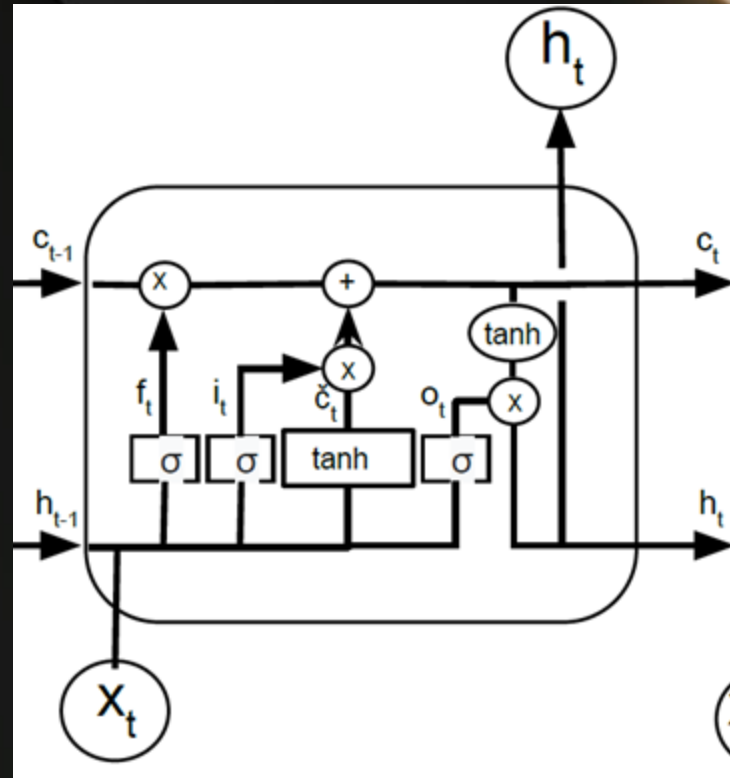
- Get new context vector.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \hat{C}_t$$

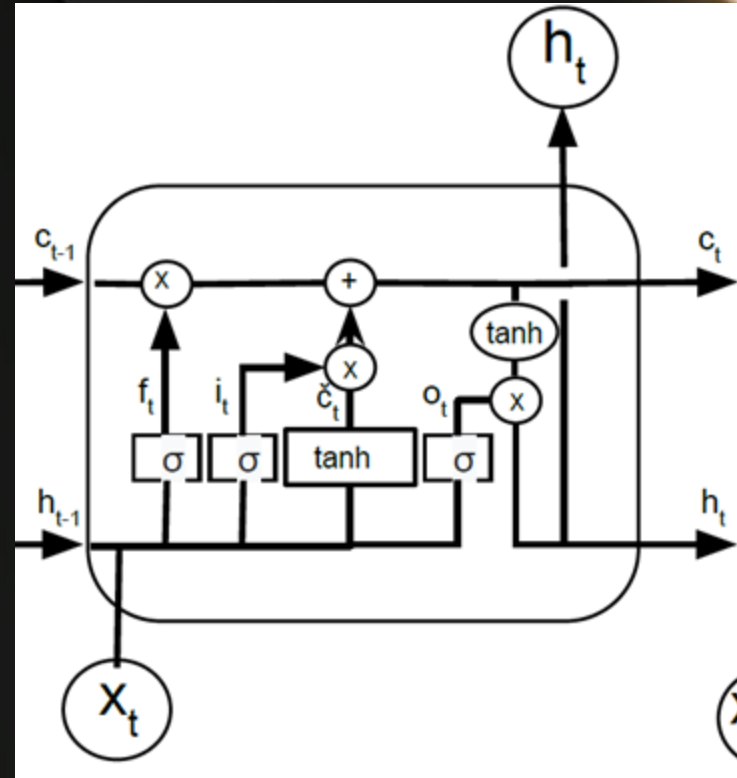


LSTM/GRU

- Output gate
 - o Decide which information is required for the current hidden state.

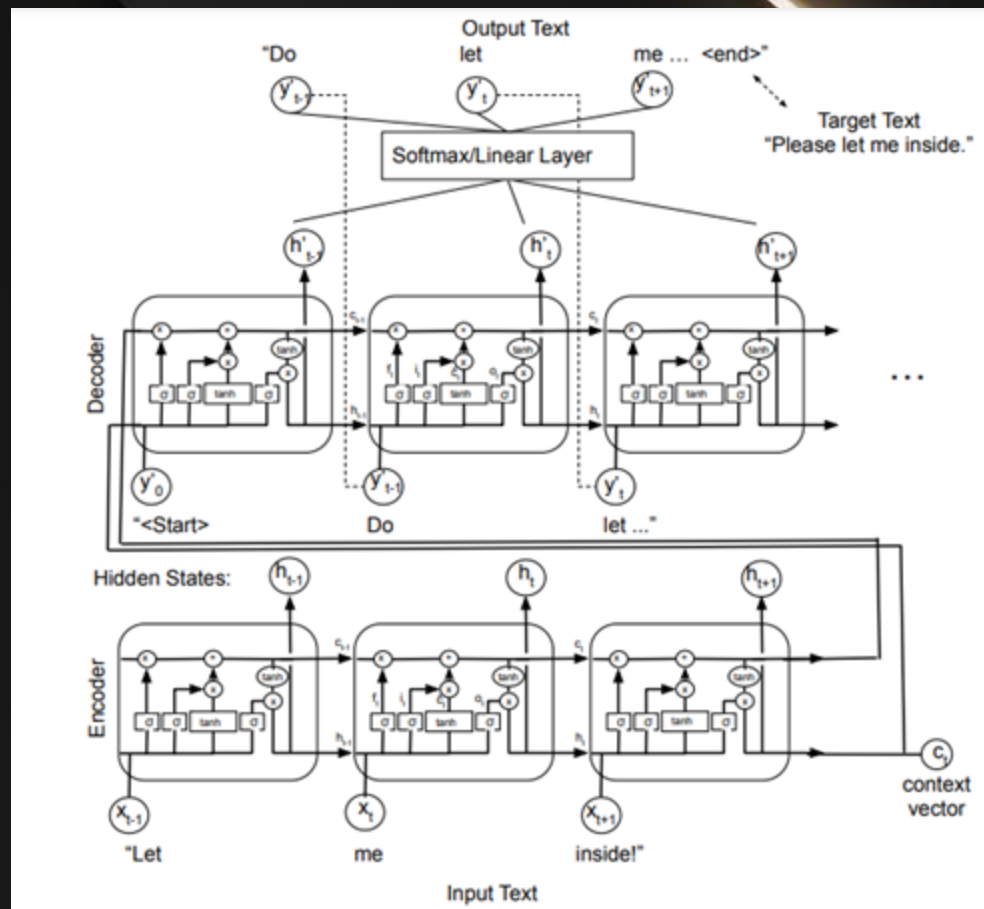
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



LSTM/GRU

- Context vector vs. Hidden State

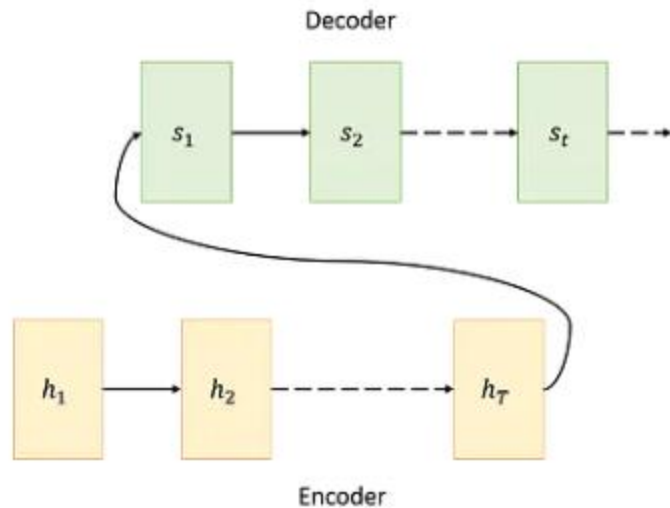


Attention

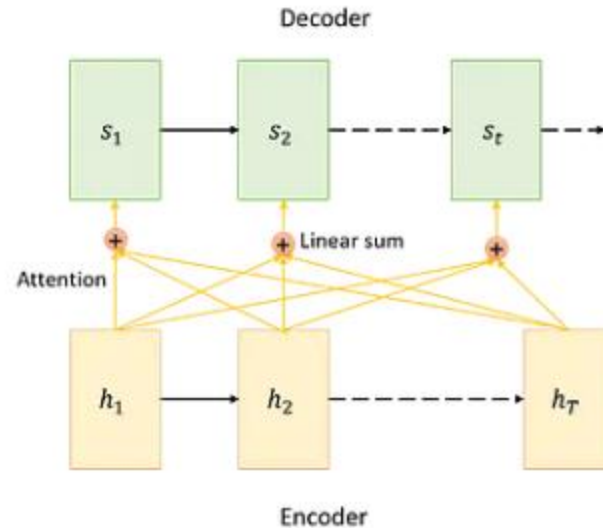
“All that is gold does not glitter,
Not all those who wander are lost;
The old that is strong does not wither,
Deep roots are not **reached** by the **frost**.”

From the ashes a fire shall be woken,
A light from the shadows shall spring;
Renewed shall be blade that was broken,
The crownless again shall be king.”

Attention mechanism helps to look at all hidden states from encoder sequence for making predictions unlike vanilla Encoder-Decoder approach.

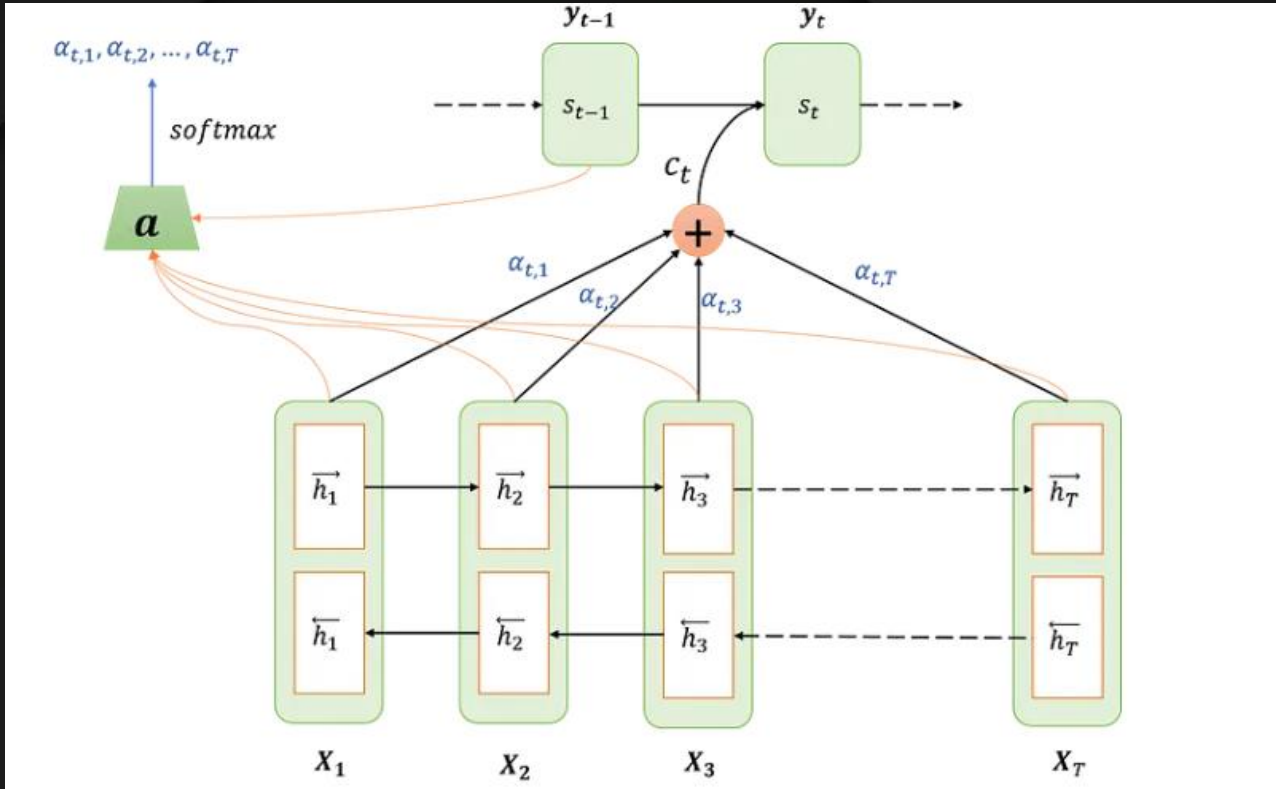


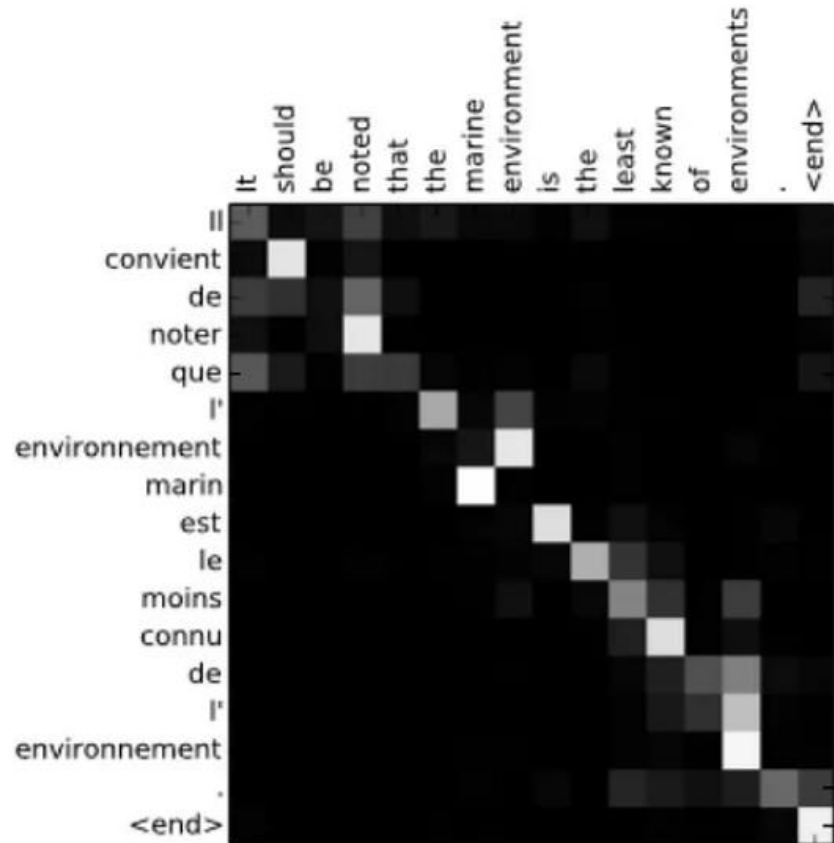
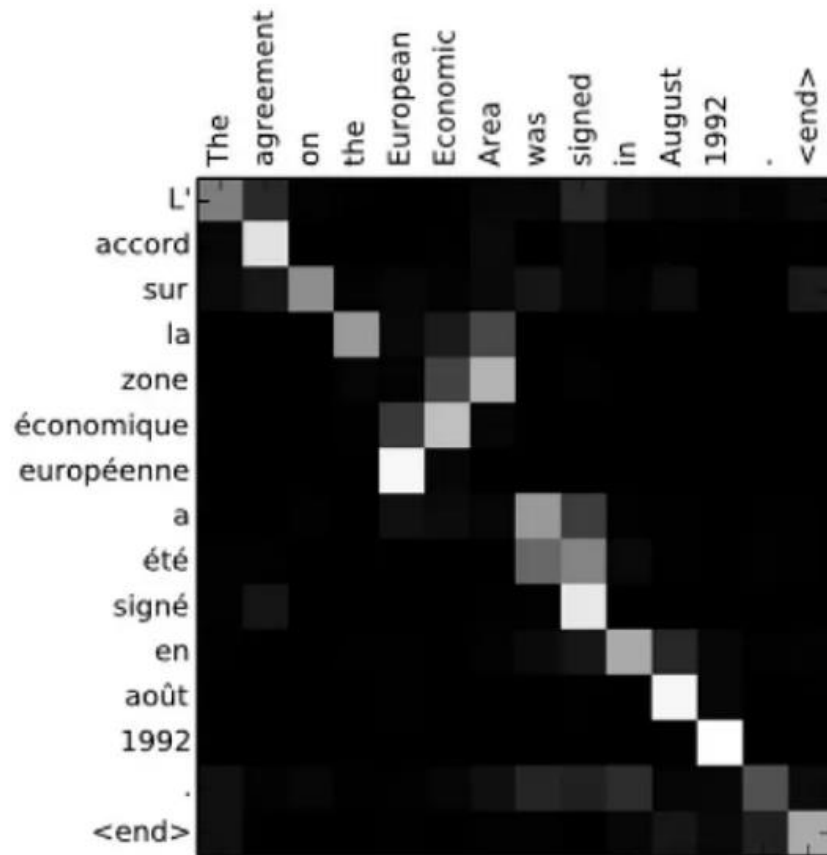
(a) Vanilla Encoder Decoder Architecture



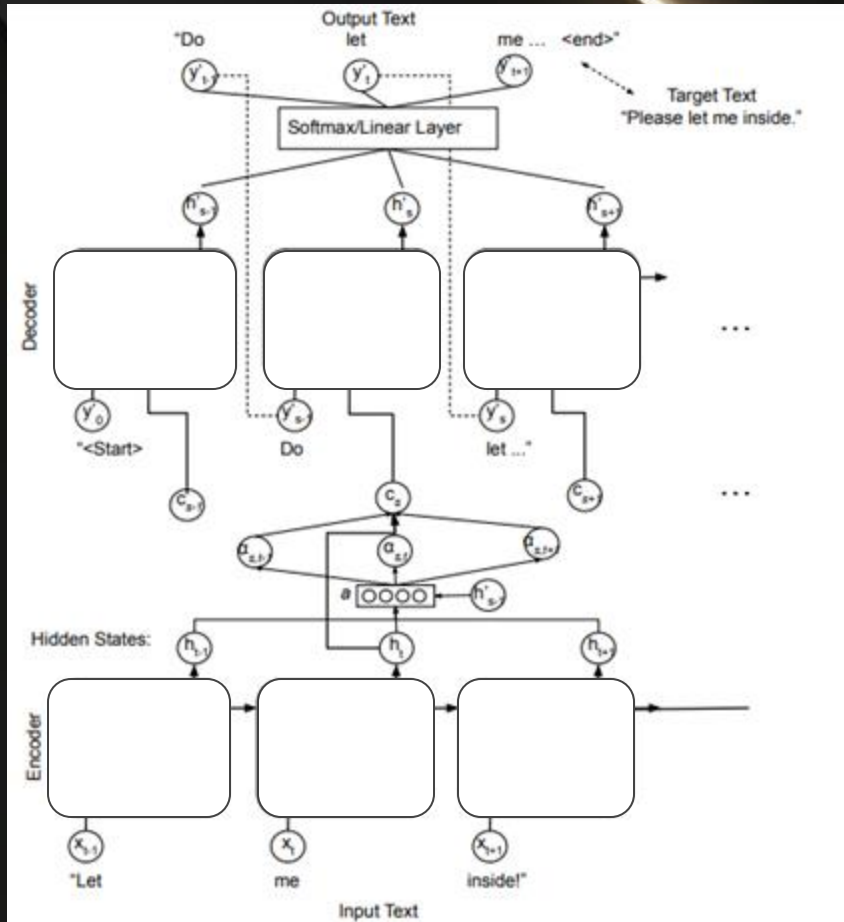
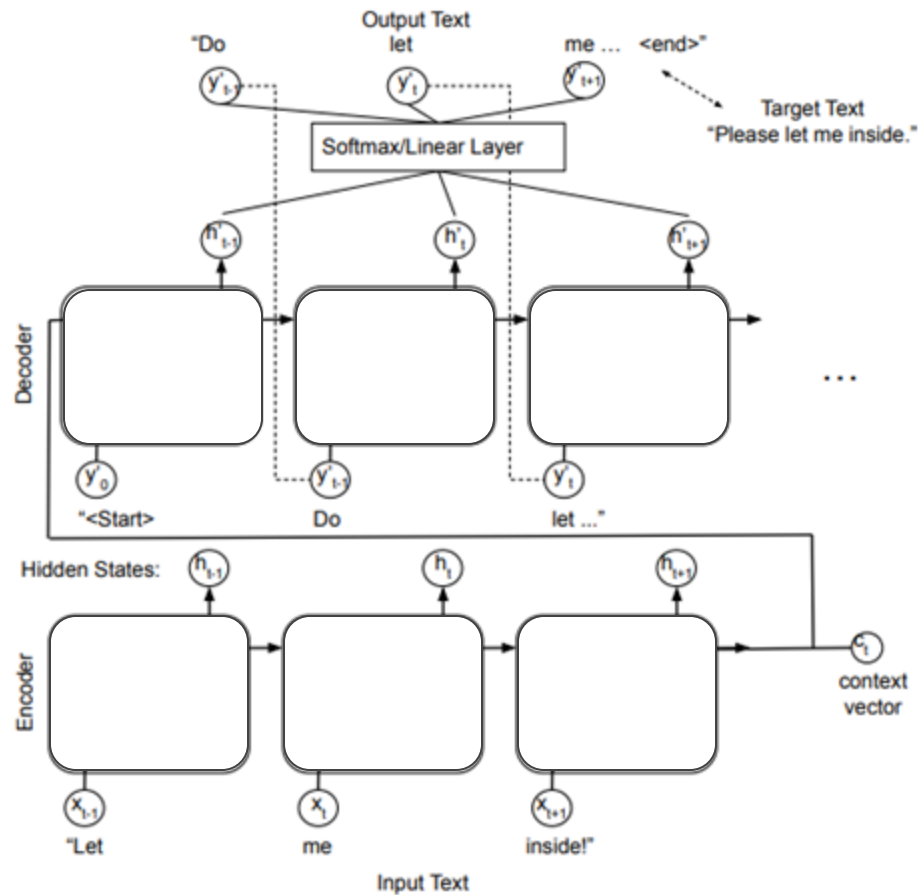
(b) Attention Mechanism

Use a Neural Network to “learn” which hidden encoder states to “attend” to and by how much.





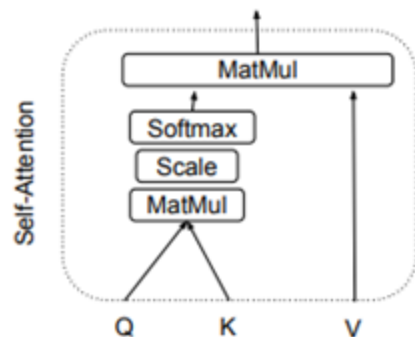
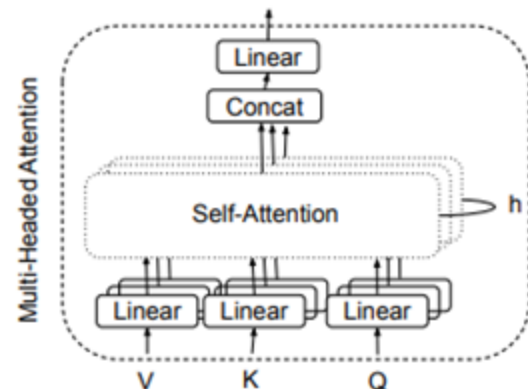
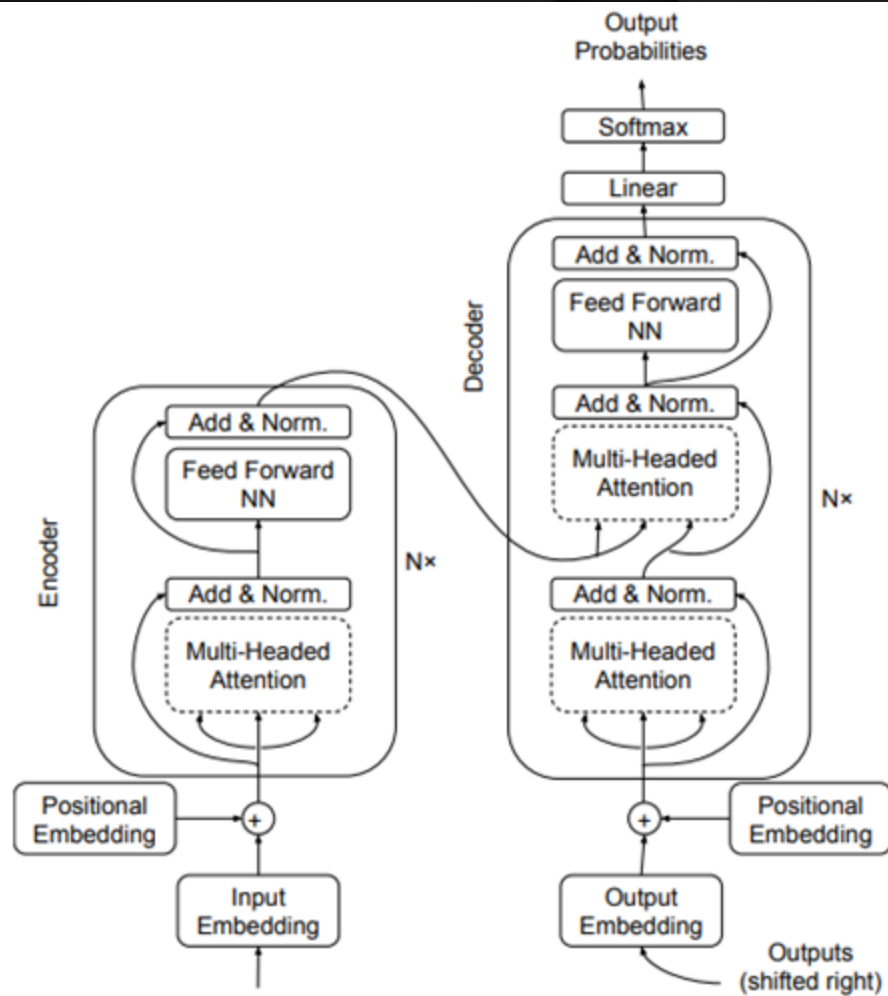
Attention



Transformers

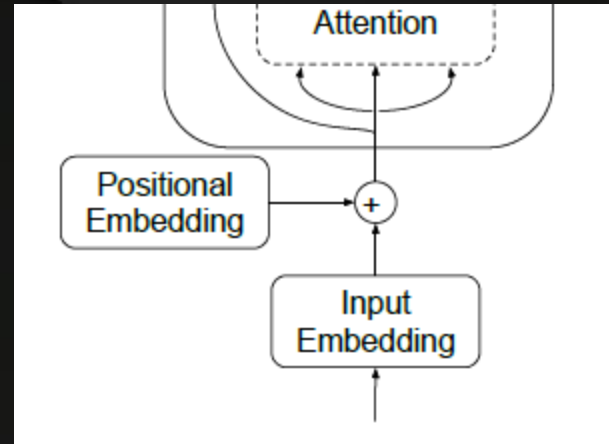
Transformers

- Introduced in 2017 (Vaswani et al.)
- Great increase in popularity
- Strong performance in NLP tasks



Positional Embeddings

- Positional information needed
- d dimensional vector (d = dimension of the token embedding)
- pos - position in input sentence
- p_t - positional encoding



$$p_t = f(t)^{(i)} := \begin{cases} \sin(pos/10000^{2i/d}) & \text{if } i = 2k \\ \cos(pos/10000^{2i/d}) & \text{if } i = 2k + 1 \end{cases}$$

Multi-headed attention

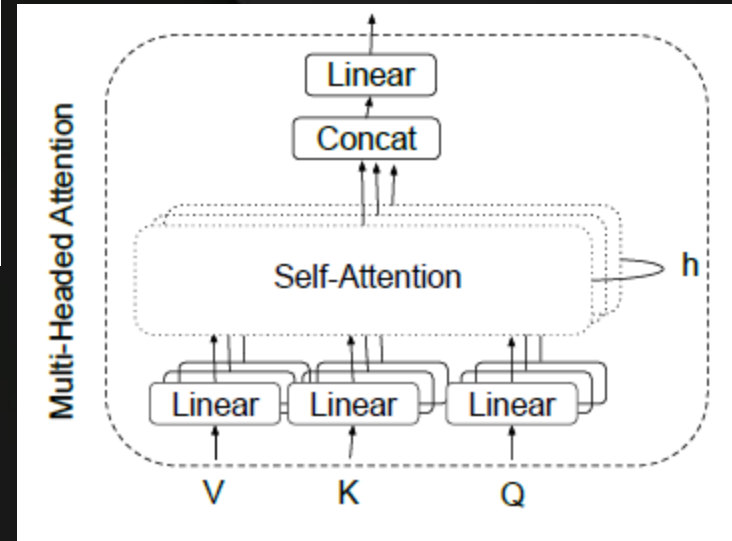
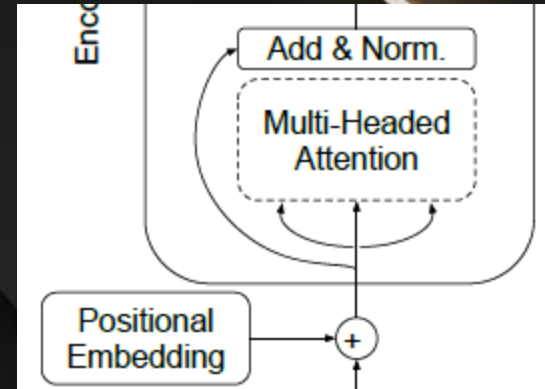
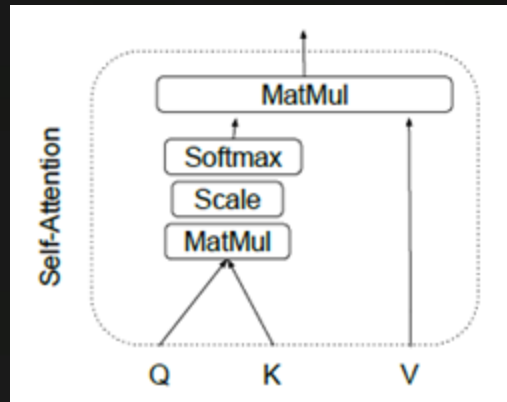
- Captures different aspects
- Query, Key, Values

$$Q = W_Q \cdot X + b_Q$$

$$K = W_K \cdot X + b_K$$

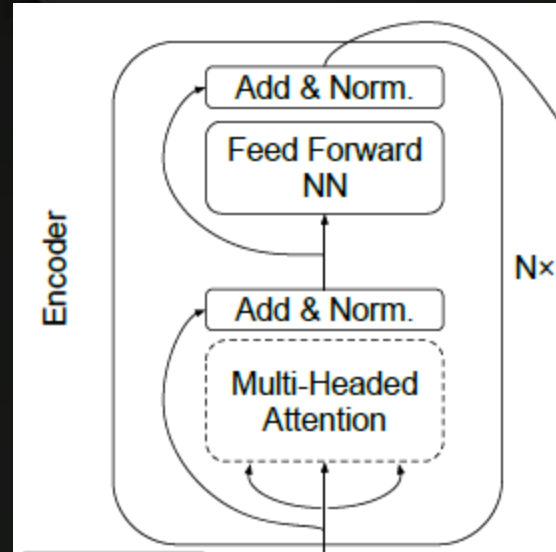
$$V = W_V \cdot X + b_V$$

$$X' = \text{softmax}(\text{scale}(Q \cdot K^T)) \cdot V$$



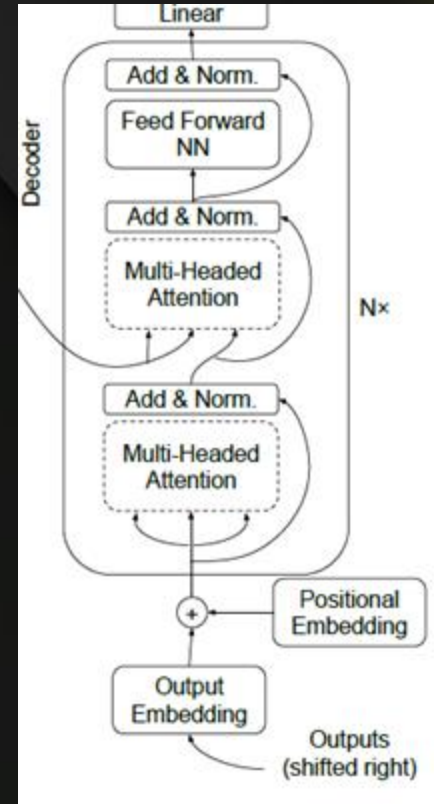
Feed Forward NN

- Two linear layers separated by a ReLu function
 - $\text{Relu}(X) = \max(0, X)$
- Output passed to another encoder
 - Repeated N times
 - “stack of N encoders”



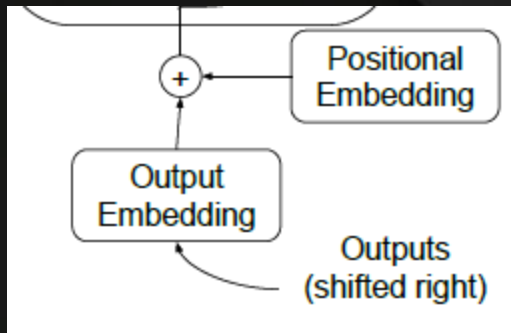
Decoder

- Autoregressive
 - Begins with a start token and combines information of previous output tokens with information from encoder
- Produces text until end token



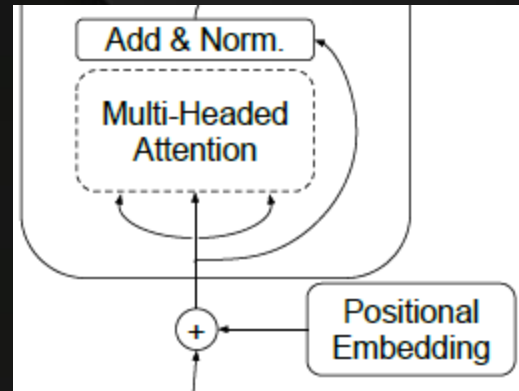
Output Embedding + Positional embedding

- Similar to encoder input positional and text embeddings



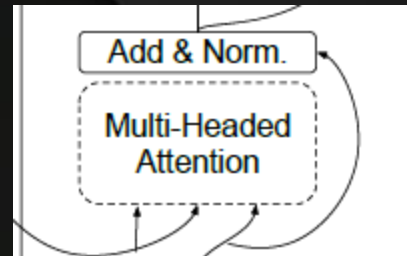
First multi-headed attention

- Similar to encoder
- Not allowed to look at future output tokens (during training)
- Look-ahead mask applied



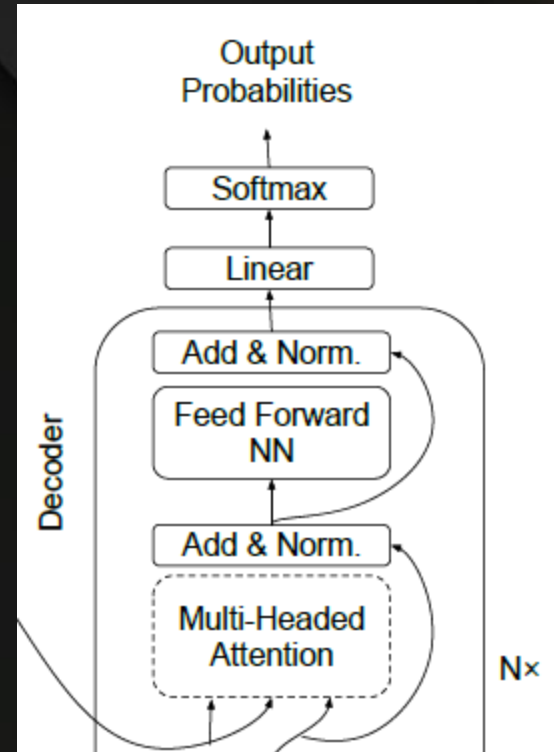
Second multi-headed attention

- Q and K come from encoded information
- Apply attention to decoder values

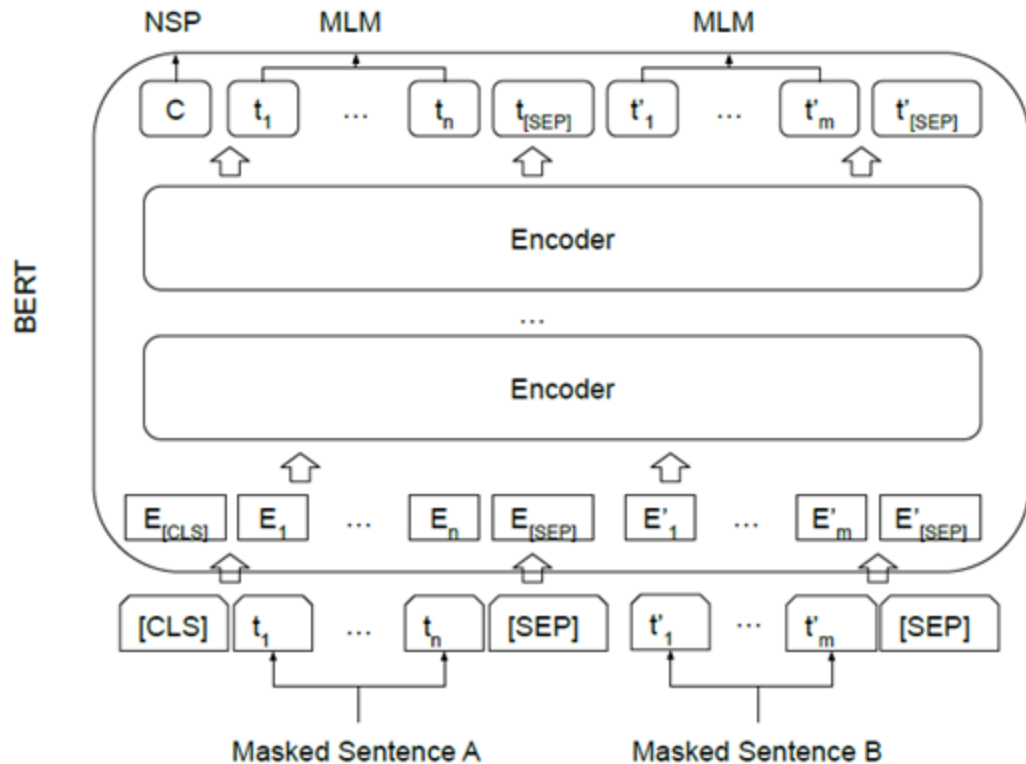


Feed forward network + Output

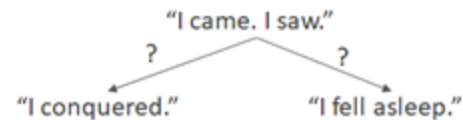
- Similar to encoder
- Decoder also stacked N times
- Final decoder output passed through linear layer, then softmax



BERT



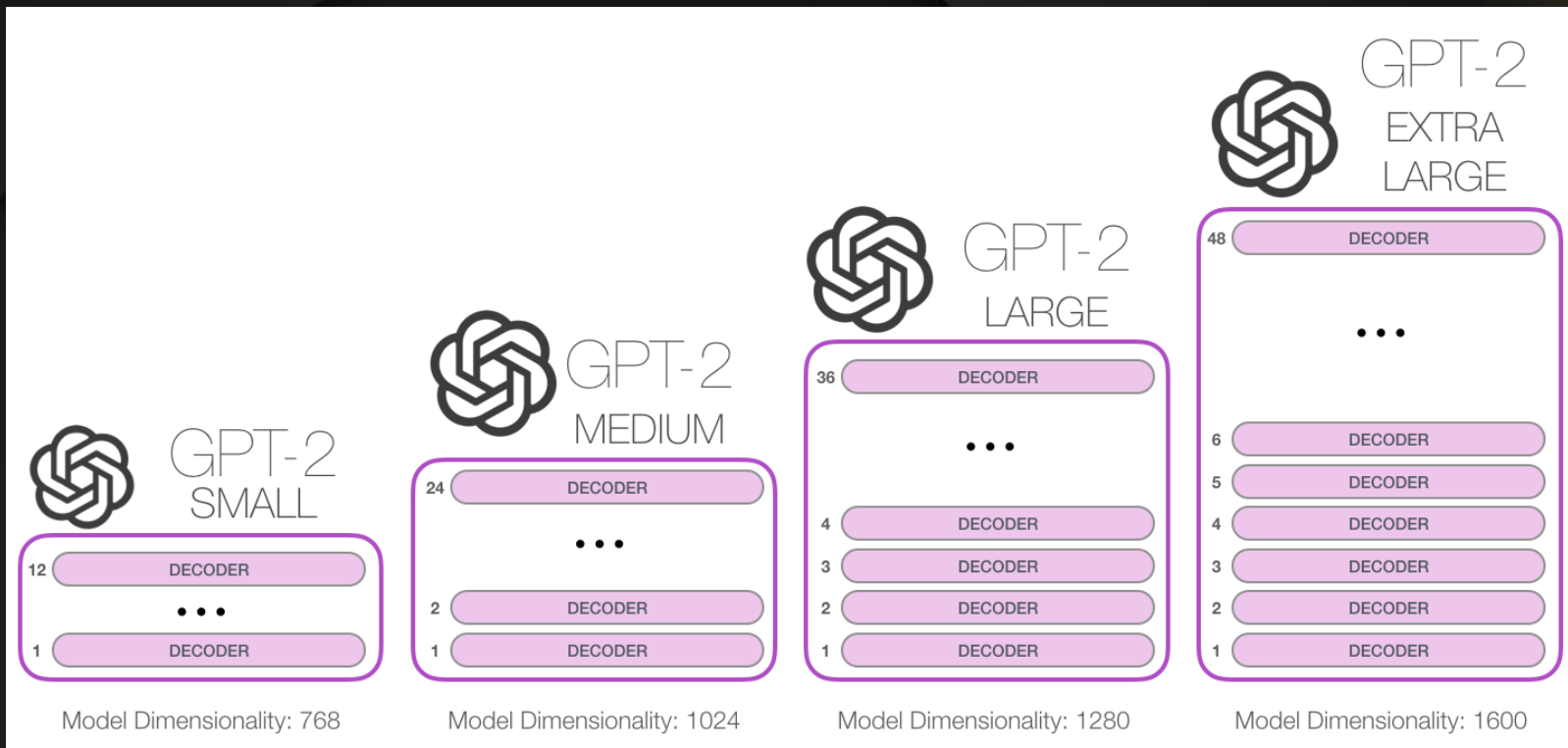
Next Sentence Prediction (NSP)



Masked Language Modeling (MLM)

"<mask> came. I <mask>. I conquered."
↓
"We came. I saw. I conquered."

GPT





Questions?